Linker

- Linker
  - Introduction
  - Learning Objectives
  - **Task 1** Interpreting compiler and linker errors
    - Keil workflow
    - Makefile workflow
  - **Task 2:** Debugging
    - Keil workflow
    - Makefile workflow
  - **Task 3:** Analysing symbols
    - Keil workflow
    - Makefile workflow
  - **Task 4:** Linker map
    - Keil workflow
    - Makefile workflow
  - Grading

Introduction

In this lab, you will learn how to fix errors that occur when using external libraries and how to configure the debugger so that it can step into library functions. Additionally, you will learn how to interpret the output files created by the assembler/compiler and linker

The project consists of a library along with the corresponding header files. You must specify the correct locations of the header files in the makefile and add any missing `#include` directives in `main.c`. Furthermore, you need to specify the library to use in the linker options.

After that, you will debug the program. You will learn the difference between libraries with and without debug symbols.

> This laboratory differs significantly between the workflow with **Keil** and the workflow with **Makefile** (VS Code uses the Makefiles). Make sure to follow the instructions for your workflow.

Learning Objectives

- You can interpret error messages from the compiler and linker and fix the problems.
- You use external libraries.
- You can extract the symbols from elf files and interpret the output.

- You can interpret linker map files.

Task 1 Interpreting compiler and linker errors

Your first task is to make the program compile. Try building the program and interpret the compiler/linker error messages to locate the problem.

Also see the task list in `main.c`.

> This application is not just divided into modules, but also a library, whose headers can be found in the directory `inc`. You need to tell your toolchain where to look for the headers.
> You also need to tell the toolchain what libaries to link against and where to find them.

See the chapters below for workflow specific instructions.

Once you fixed all errors, you can flash the program to the CT board. The functionality is quite simple: Pressing the button T0 inverts the LEDs. On the first activation, the LEDs are set to a predefined pattern.

Keil workflow

Add the path to the include files in the project settings under the tab `C/C++`.

Configure the library paths and names under the tab `linker` in the project settings in the field `Misc Controls`.

Makefile workflow

To add the include path, extend the variable `INC_DIR`.

In order to fix the linker error, you first need to build the library. Use the makefile in the directory `libctio` to build an archive with `make rebuild`. Follow the example of `libctboard` to add `libctio` to the linker command.

> Add elements to a variable with `key += value`. Explore the file `<ct-root-directory>/common/root.mk` to understand how the variables `LIB_PATHS` and `LIB_NAMES` are used in the linker command.

> Note that GCC automatically adds a 'lib' to library names.

Task 2: Debugging

Try to step through the program line by line with the debugger. Observe the behaviour of the debugger on functions (`read8`, `write8`, `toggle`).

Keil workflow

> Use the button `Step`, or the function key `F11` to step through the program line by line.

First debug the program linked against `lib\read_write.lib`. What do you observe?

Now link the program against `lib_debug\read_write.lib` (change the entry in the `Misc Controls` field in the linker tab). Do not forget to rebuild the program. What changed?

At last link the program against `lib_debug_with_src\read_write.lib`. After you started the debugger you need to tell it, where to find the sources. To do so, enter the following command in the debuggers *command window*.

```
set src = C:\<path-to-laboratory>\keil\lib_debug_with_src
```

Keil can not handle spaces in the path, if your user name contains spaces, you will need to relocate the laboratory to a directory with no spaces in its path.

What do you observe and what is the cause for the different behaviour?

Makefile workflow

> If using `GDB` directly use the command `step` or `s` to debug the program line for line.
> An empty command (just hitting enter) will execute the last used command.

Make sure you built `libctio` without debug symbols (make targets `all`, `build` or `rebuild` in directory `libctio/`). Rebuild your program (`make rebuild` in current directory).

Debug the program and document your observation.

Now recompile the library (libctio) with debug symbols. Use the make target `debug` in the directory `libctio/`. Do not forget to rebuild your program.

Debug the program. What changed and what is the cause for the different behaviour?

Task 3: Analysing symbols

For this task you will analyse the symbols of object files and binaries.

Keil workflow

The tool `fromelf.exe` can print the contents of ELF (executable and linkable format) files in a human readable format. Object (`.o`) and library (`.lib`) files and binaries (`.axf`) are all ELF files.

```
Command Prompt

C:\Temp>C:\KEIL_v5\ARM\ARMCLANG\bin\fromelf.exe
Product: MDK-ARM Lite 5.40
Component: Arm Compiler for Embedded 6.22
Tool: fromelf [5ee90f00]

ARM image conversion utility
fromelf [options] input_file

Options:
       --help         display this help screen
       --vsn          display version information
       --output file  the output file. (defaults to stdout for -text format)
       --nodebug      do not put debug areas in the output image
       --nolinkview   do not put sections in the output image

Binary Output Formats:
       --bin          Plain Binary
       --m32          Motorola 32 bit Hex
       --i32          Intel 32 bit Hex
       --vhx          Byte Oriented Hex format

       --base addr    Optionally set base address for m32,i32

Output Formats Requiring Debug Information
       --fieldoffsets Assembly Language Description of Structures/Classes
       --expandarrays Arrays inside and outside structures are expanded

Other Output Formats:
       --elf          ELF
       --text         Text Information

                Flags for Text Information
                -v          verbose
                -a          print data addresses (For images built with debug)
                -c          disassemble code
                -d          print contents of data section
                -e          print exception tables
                -g          print debug tables
                -r          print relocation information
                -s          print symbol table
                -t          print string table
                -y          print dynamic segment contents
                -z          print code and data size information
```

Extract the symbol table from `Objects\toggle.o`, `Objects\main.o` and `lib\read_write.lib`. Ignore the debug information and focus code and data symbols.

What attributes indicate that a symbol is local, exported or imported? Insert your answers in the table below.

local symbols	imported symbols	exported symbols

If you miss the header files for a library, could you write the headers yourself with the extracted symbol table?

Makefile workflow

The tool `arm-none-eabi-readelf` can print the contents of ELF (executable and linkable format) files in a human readable format. Object (`.o`) and library files (`.a`) and binaries (`.axf`) are all ELF files.

Use the command below to extract the symbol tables from the files `obj/main.o`, `obj/toggle.o` and `libctio/libctio.a`.

```
arm-none-eabi-readelf -s <path/to/elf>

# example
arm-none-eabi-readelf -s obj/main.o
```

What attributes indicate that a symbol is local, exported or imported? Insert your answers in the table below.

local symbols	imported symbols	exported symbols

If you miss the header files for a library, could you write the headers yourself with the extracted symbol table?

Task 4: Linker map

The linker command we use, instructs the linker to generate a map file.

Keil workflow

Analyse the map file. Explain the memory map using the map file. Search for the 'memory map' section. Where is the vector table, data, constants, code, etc. located?

Makefile workflow

Analyse the map file. Explain the memory map using the map file. Search for the 'memory map' section. Where is the vector table, data, constants, code, etc. located?

Grading

The working programs have to be presented to the lecturer. The student has to understand the solution / source code and has to be able to explain it to the lecturer.

Criteria	Weight
Task 1 completed. program compiles and behaves as described	1/4
Student answered the questions and is able explain their answers for Task 2.	1/4
Student answered the questions and is able explain their answers for Task 3.	1/4
Student answered the questions and is able explain their answers for Task 4.	1/4